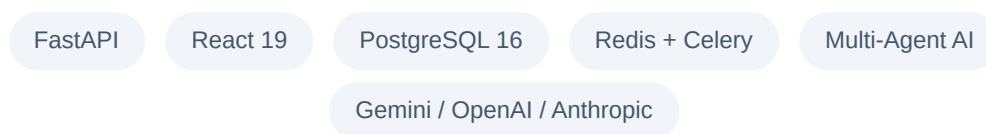


Codalyra-AI

Agentic PR Review Pipeline — Project Documentation



A multi-agent code review system where 4 specialized AI agents analyze a code diff from different angles, then a synthesis agent combines their findings into a unified, prioritized report — with a built-in single-prompt baseline for measurable comparison.

Author: Hamza Haji (HMZ-H)

Repository: github.com/HMZ-H/codalyra-ai

Generated: September 2026

Table of Contents

1. Project Overview & Architecture

2. Directory Structure

3. Backend — Folder-by-Folder Breakdown

4. Frontend — Folder-by-Folder Breakdown

5. The Review Pipeline — End-to-End Workflow

6. Data Flow & Component Interaction

7. Database Schema & Models

8. Authentication & Security

9. GitHub Integration

10. Advanced Features

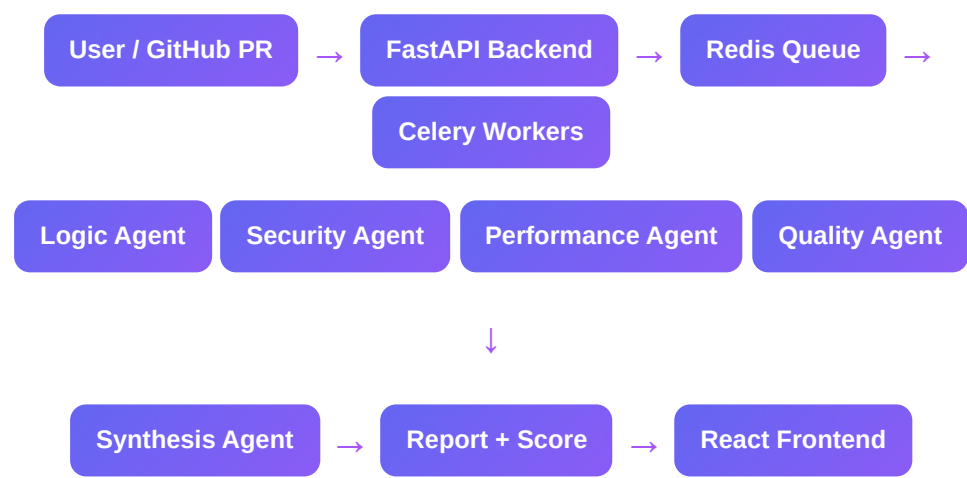
11. DevOps & Deployment

12. Testing Strategy

1. Project Overview & Architecture

Codalyra-AI is a full-stack, multi-agent code review platform. Users submit a code diff (pasted or fetched from a GitHub PR), and the system dispatches it to **4 specialist AI agents** that each analyze the code from a different perspective. A **synthesis agent** then merges, deduplicates, and prioritizes all findings into a single report. A **baseline single-pass review** runs in parallel for quality comparison.

High-Level Architecture



Tech Stack

Layer	Technology	Purpose
Backend API	FastAPI (Python 3.12)	REST API, WebSocket, async OAuth
ORM	SQLAlchemy 2.0	Database models & queries
Database	PostgreSQL 16	Persistent storage
Task Queue	Redis 7 + Celery	Async review pipeline execution
AI / LLM	Gemini Flash (default), OpenAI, Anthropic	Multi-provider code analysis
Migrations	Alembic	Database schema versioning
Frontend	React 19 + Vite	Single-page application
Styling	Custom CSS	Dark/light theme, responsive
Auth	JWT + bcrypt + GitHub OAuth	User authentication
Encryption	Fernet (via SECRET_KEY)	API key storage at rest

Containerization	Docker + Docker Compose	5-service deployment
CI/CD	GitHub Actions	Lint, test, build on push/PR

2. Directory Structure

```
codalyra-ai/
├── backend/ - FastAPI application
│   ├── alembic/ - Database migrations
│   │   └── versions/ - 12 migration scripts
│   ├── app/ - Main application package
│   │   ├── ai/ - LLM clients, prompts, analysis
│   │   ├── api/v1/ - REST API route handlers
│   │   ├── core/ - Security, JWT, encryption, permissions
│   │   ├── database/ - SQLAlchemy models & session
│   │   │   └── models/ - 12 ORM models
│   │   ├── github/ - GitHub diff parsing utilities
│   │   ├── middleware/ - Rate limiting
│   │   ├── repositories/ - Data access layer
│   │   ├── schemas/ - Pydantic request/response models
│   │   ├── scripts/ - Admin, seed, evaluation scripts
│   │   ├── services/ - Business logic layer
│   │   ├── tests/ - 126 pytest tests
│   │   ├── utils/ - Constants, helpers, logger
│   │   ├── validators/ - Static analysis & custom rules
│   │   ├── workers/ - Celery task definitions
│   │   ├── main.py - FastAPI app factory
│   │   ├── config.py - Settings via pydantic-settings
│   │   └── dependencies.py - JWT auth dependency injection
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ruff.toml - Linter configuration
├── frontend/ - React 19 + Vite SPA
│   ├── src/
│   │   ├── api/ - Axios HTTP client
│   │   ├── components/ - Reusable UI components
│   │   ├── context/ - React context (auth state)
│   │   ├── hooks/ - Custom hooks (WebSocket, theme)
│   │   ├── pages/ - 14 page components
│   │   ├── styles/ - Global CSS
│   │   └── __tests__/ - 47 frontend tests
│   ├── Dockerfile
│   └── nginx.conf - Reverse proxy config
├── docker-compose.yml - 5 services
└── .github/workflows/ci.yml - CI pipeline
```

3. Backend — Folder-by-Folder Breakdown

3.1 `app/ai/` — AI & LLM Layer

The brain of the system. Contains everything related to communicating with LLMs and processing their outputs.

File	Responsibility
<code>prompts.py</code>	Defines system prompts for all 6 agents: logic (correctness, edge cases), security (OWASP, injection, secrets), performance (complexity, memory), quality (style, naming, DRY), synthesis (merge & deduplicate), and baseline (single-pass comparison). Each prompt enforces a strict JSON schema for findings.
<code>client.py</code>	Original Gemini-only <code>LLMClient</code> . Provides <code>chat()</code> for raw text and <code>chat_json()</code> for JSON-mode responses. Includes retry logic with exponential backoff. Used as the default when no alternative provider is configured.
<code>providers.py</code>	Multi-provider abstraction. Abstract <code>LLMProvider</code> base class with concrete implementations for Gemini , OpenAI , and Anthropic . Factory function <code>create_provider(name, api_key, model)</code> instantiates the right one. All providers implement <code>chat_json()</code> and return parsed data with token counts.
<code>reviewer.py</code>	<code>AgentReviewer</code> orchestrates the actual LLM calls. Methods: <code>run_agent()</code> for specialist reviews (prepends static findings), <code>run_synthesis()</code> to merge all specialist outputs, and <code>run_baseline()</code> for a single-pass review. Returns findings with metadata (tokens, duration).
<code>analyzer.py</code>	Post-processing utilities: deduplication by (file, line, category), severity-based scoring (critical: -30, warning: -10, info: -2 from a base of 100), sorting, category extraction, and baseline comparison metrics.
<code>fixer.py</code>	<code>AutoFixer</code> takes a diff + findings, filters to critical/warning items, and asks the LLM to generate corrected code snippets with explanations. Returns <code>{original_code, fixed_code, explanation}</code> patches.
<code>embeddings.py</code>	Placeholder for future embedding-based similarity search.

Key Design Decision

Each specialist agent receives the diff **plus** relevant static analysis findings as pre-context, so the LLM can validate, extend, or override the static checks with deeper semantic understanding.

3.2 `app/api/v1/` — REST API Routes

All endpoints are versioned under `/api/v1/`. The `__init__.py` registers all 18 routers.

Router File	Prefix	Endpoints
<code>auth.py</code>	<code>/auth</code>	Register, login, /me, GitHub OAuth login + connect
<code>projects.py</code>	<code>/projects</code>	CRUD for projects
<code>reviews.py</code>	<code>/reviews</code>	Create review, list, get, report, auto-fix
<code>github.py</code>	<code>/github</code>	List repos, pulls, get diff, post comments, webhook handler
<code>settings.py</code>	<code>/settings</code>	API key management (store/delete encrypted keys)
<code>analytics.py</code>	<code>/analytics</code>	Overview, score trends, category stats, agent performance
<code>teams.py</code>	<code>/teams</code>	Team CRUD + member management with roles
<code>custom_rules.py</code>	<code>/projects/{id}/rules</code>	CRUD for per-project regex-based custom rules
<code>feedback.py</code>	<code>/feedback</code>	Submit finding feedback, get stats, agent accuracy
<code>agent_configs.py</code>	<code>/projects/{id}/agents</code>	Configure provider/model/enabled per agent per project
<code>health.py</code>	<code>/health</code>	Health check + database connectivity
<code>exports.py</code>	<code>/exports</code>	Markdown report export
<code>ws.py</code>	<code>/ws</code>	WebSocket for real-time review status updates
<code>users.py</code>	<code>/users</code>	User profile management
<code>repositories.py</code>	<code>/repositories</code>	CRUD for connected repositories
<code>tasks.py</code>	<code>/tasks</code>	CRUD for review tasks

<code>runs.py</code>	<code>/runs</code>	CRUD for individual agent runs
<code>evaluations.py</code>	<code>/evaluations</code>	Read evaluation results (findings, scores)
<code>trajectories.py</code>	<code>/trajectories</code>	Read agent execution traces

3.3 `app/core/` — Security & Infrastructure

File	Responsibility
<code>jwt.py</code>	Creates and verifies JWT access tokens (HS256, 30-min expiry by default)
<code>security.py</code>	bcrypt password hashing and verification
<code>encryption.py</code>	Fernet symmetric encryption for storing API keys. Derives a key from <code>SECRET_KEY</code> via SHA-256.
<code>permissions.py</code>	Ownership verification guards. Checks that the requesting user owns the project, review, task, run, or repository. Raises <code>NotFoundException</code> or <code>ForbiddenException</code> on failure.
<code>exceptions.py</code>	Custom exception classes: <code>CodalyraException</code> , <code>ConflictException</code> , <code>UnauthorizedException</code> , <code>NotFoundException</code> , <code>ForbiddenException</code> .

3.4 `app/database/` — Data Layer

`base.py` declares the SQLAlchemy `Base` class. `session.py` creates the engine and provides a `get_db()` dependency that yields a session per request.

Models (12 tables)

Model	Table	Key Fields
<code>User</code>	<code>users</code>	email, username, hashed_password, github_id/token, encrypted API keys (gemini, openai, anthropic)
<code>Project</code>	<code>projects</code>	name, owner_id, team_id, slack/discord webhook URLs
<code>Repository</code>	<code>repositories</code>	project_id, name, url, clone_url, default_branch
<code>Review</code>	<code>reviews</code>	project_id, diff_content, status, overall_score, summary, findings_count, baseline comparison, metadata (JSON)
<code>Task</code>	<code>tasks</code>	project_id, review_id, type, status
<code>Run</code>	<code>runs</code>	task_id, agent_type, status, started/completed_at

Evaluation	evaluations	run_id, score, findings (JSON), token_usage, duration
Trajectory	trajectories	run_id, step_number, action, input, output (execution trace)
AgentConfig	agent_configs	project_id, agent_type, is_enabled, provider, model_name, custom_prompt
Team	teams	name, slug, owner_id. Members via TeamMember (admin/reviewer/viewer roles)
CustomRule	custom_rules	project_id, name, regex pattern, severity, category, file_pattern filter, is_enabled
FindingFeedback	finding_feedback	review_id, finding_hash (SHA-256), action (accept/dismiss/false_positive), agent, category

3.5 app/services/ — Business Logic

Service	Responsibility
auth_service.py	Register, login (email/password), GitHub OAuth login (exchange code → get user → create/link account), GitHub account connection for existing users.
review_service.py	Core orchestrator. Creates Review + Task + 6 Run records, dispatches Celery tasks for each agent (skipping disabled ones), assembles the final report with baseline comparison, and triggers auto-fix generation.
report_service.py	Assembles a structured report by collecting findings from each agent run's evaluation, grouping by agent type, computing overall score.
analytics_service.py	Dashboard data: paginated review history, daily score trends, category breakdowns (most common findings), per-agent performance (avg score, duration, finding counts), and user overview.
feedback_service.py	Records user feedback on findings (upsert by finding_hash), computes per-category dismiss rates and per-agent accuracy metrics.
notification_service.py	Sends Slack Block Kit and Discord embed notifications on review completion with score, severity breakdown, and summary.
github_service.py	GitHub API client: OAuth token exchange, user info, list repos, list PRs, get PR diff, post review comments and inline comments.

<code>export_service.py</code>	Generates Markdown-formatted review reports for download/export.
<code>project_service.py</code>	Project CRUD with ownership checks.
<code>execution_service.py</code>	Manages task execution state transitions.

3.6 `app/validators/` — Static Analysis

Regex-based static analysis runs *before* the LLM, providing initial findings that the AI agents can validate and extend.

Validator	What It Detects
<code>detector.py</code>	Language detection from diffs. Parses file headers, maps extensions to languages, returns by frequency.
<code>security_validator.py</code>	Hardcoded secrets (API keys, AWS keys, GitHub/Slack tokens), dangerous functions (eval, exec, os.system, pickle, innerHTML/XSS).
<code>python_validator.py</code>	Python-specific: bare excepts, mutable default arguments, assert in production, wildcard imports, TODO comments, print statements.
<code>javascript_validator.py</code>	JS/TS-specific: console.log, var usage, == instead of ===, alert(), document.write(), eval().
<code>golang_validator.py</code>	Go-specific: ignored errors (<code>_ = err</code>), fmt.Println, panic usage, empty interface.
<code>custom_rules.py</code>	Runs user-defined regex rules from the database. Checks file patterns and line patterns against added lines in the diff.

3.7 `app/workers/` — Celery Task Definitions

File	Tasks
<code>celery_app.py</code>	Celery app configuration: Redis broker/backend, JSON serialization, UTC timezone, task module autodiscovery.
<code>review_tasks.py</code>	The core pipeline: <code>run_specialist_review</code> (static analysis + LLM per agent), <code>check_and_trigger_synthesis</code> (waits for all 4 specialists), <code>run_synthesis_review</code> (merges findings), <code>run_baseline_review</code> (single-pass comparison). Records trajectories, evaluations, handles errors/retries. Dispatches notifications on completion.
<code>github_tasks.py</code>	Background tasks for GitHub operations (post comments, etc.).

<code>execution_tasks.py</code>	Generic task execution management.
<code>evaluation_tasks.py</code>	Evaluation scoring tasks.
<code>report_tasks.py</code>	Report generation tasks.

3.8 Other Backend Folders

Folder	Purpose
<code>app/repositories/</code>	Data access layer implementing the Repository pattern. Provides <code>UserRepository</code> , <code>ProjectRepository</code> , <code>ReviewRepository</code> , <code>RunRepository</code> , <code>TaskRepository</code> for clean database access.
<code>app/schemas/</code>	Pydantic models for request validation and response serialization. Includes validation logic (e.g., diff size limits, regex pattern validation, role enums).
<code>app/middleware/</code>	<code>rate_limit.py</code> — Redis-backed sliding-window rate limiter (10 req/hr per action). Gracefully degrades when Redis is unavailable.
<code>app/scripts/</code>	CLI utilities: <code>create_admin.py</code> (admin user), <code>seed_database.py</code> (test data), <code>run_evaluation.py</code> (benchmark agent vs. baseline performance).
<code>app/github/</code>	Low-level GitHub utilities: diff parsing, webhook signature verification.
<code>alembic/</code>	12 migration scripts covering the full schema evolution from initial tables through teams, webhooks, custom rules, and feedback.

4. Frontend — Folder-by-Folder Breakdown

4.1 src/api/ — HTTP Client

`client.js` — Centralized Axios client. Configures the base URL (`/api/v1` by default, production-ready behind nginx). Includes request interceptor (attaches JWT from `localStorage`) and response interceptor (auto-redirects to `/login` on 401). Exports 14 API modules:

- `auth` — register, login, me, GitHub callback/connect
- `projects` , `repositories` , `tasks` , `runs` — standard CRUD
- `reviews` — create, list, get, report, autoFix
- `github` — listRepos, connectRepo, listPulls, getPullDiff, reviewPull, postComments
- `settings` — API key management
- `analytics` — overview, reviews, scoreTrends, categories, agents
- `agentConfigs` — list, update, reset, providers
- `teams` — CRUD + member management
- `feedback` — submit, getForReview, getStats
- `customRules` — list, create, update, delete
- `health` — check, db

4.2 src/context/ — State Management

`AuthContext.jsx` — React Context providing authentication state across the app. On mount, checks `localStorage` for a JWT and validates it via `/auth/me` . Exposes `user` , `loading` , `login()` , `loginWithGithub()` , `logout()` , and `refreshUser()` .

4.3 src/hooks/ — Custom Hooks

Hook	Purpose
<code>useReviewSocket.js</code>	WebSocket hook for real-time review status updates. Connects to <code>/ws/reviews/{id}</code> , receives status changes and agent completion events, triggers UI refresh on completion.
<code>useTheme.js</code>	Dark/light theme toggle with <code>localStorage</code> persistence.

4.4 src/components/ — Reusable UI Components

Component	Purpose
Navbar.jsx	Navigation bar with links to Dashboard, GitHub, Analytics, Teams. Shows user avatar, theme toggle, logout.
ProtectedRoute.jsx	Route guard that redirects to /login if not authenticated.
ReviewSubmit.jsx	Modal form for creating a review: select project, paste diff or enter PR title.
ScoreTrendChart.jsx	SVG line chart showing score trends over time.
CategoryBreakdown.jsx	Horizontal bar chart of finding categories.
AgentPerformance.jsx	Cards showing per-agent metrics (avg score, duration, findings).
AuthBackground.jsx	Animated gradient background for login/register pages.

4.5 src/pages/ — Page Components (14 pages)

Page	Route	Description
Login.jsx	/login	Email/password login + GitHub OAuth button
Register.jsx	/register	User registration form
Dashboard.jsx	/dashboard	Project list, health status, create project/review modals
ProjectDetail.jsx	/projects/:id	Project overview with repos, reviews, tasks, webhook config, links to agent config & custom rules
ReviewDetail.jsx	/reviews/:id	Full review results: score, findings by severity, agent tabs, baseline comparison, auto-fix tab, finding feedback (accept/dismiss/false positive)
ReviewHistory.jsx	/reviews	Paginated review list with filters
GitHubRepos.jsx	/github	Browse connected GitHub repos, list PRs, trigger reviews from PRs
GitHubCallback.jsx	/auth/github/callback	OAuth callback handler — login or connect flow based on state param
Settings.jsx	/settings	API key management (Gemini/OpenAI/Anthropic), GitHub connection

Analytics.jsx	/analytics	Dashboard with score trends, category breakdown, agent performance charts
AgentConfig.jsx	/projects/:id/agents	Enable/disable agents, set provider & model per agent, custom prompts
Teams.jsx	/teams	Create teams, manage members (add/remove, role assignment)
CustomRules.jsx	/projects/:id/rules	Create/edit regex-based custom rules with pattern testing
Workers.jsx	/workers	Celery worker status monitoring

5. The Review Pipeline — End-to-End Workflow

This is the core workflow of the entire application. Here is exactly what happens when a user submits a code diff for review:

Step 1: Review Submission

Trigger: `POST /api/v1/reviews/`

The user submits a diff (pasted text or fetched from a GitHub PR).

`ReviewService.create_review()` is called.

- A **Review** record is created with status `"pending"`
- A **Task** record is created (type: `"review"`)
- **6 Run records** are created: one per agent (logic, security, performance, quality, synthesis, baseline)
- The system checks `AgentConfig` for the project — disabled agents are skipped
- **4 specialist Celery tasks + 1 baseline task** are dispatched in parallel

Step 2: Static Analysis (per specialist)

Celery Task: `run_specialist_review(run_id, review_id, agent_type)`

1. **Language detection:** Parse the diff headers to identify languages (Python, JS, Go, etc.)
2. **Static validators run:** Security validator (all languages) + language-specific validators + custom rules for the project
3. Static findings are filtered to those relevant to this agent's domain (e.g., security agent gets secret-detection findings)
4. Trajectory recorded: step 1 = static analysis results

Step 3: LLM Agent Analysis (per specialist)

5. Check `AgentConfig` for custom provider/model, otherwise use user's Gemini key + default model
6. `AgentReviewer.run_agent()` is called with the diff + static findings + agent-specific system prompt

7. The LLM receives the code diff, the pre-existing static findings, and the domain-specific prompt
8. Response is parsed as JSON → list of findings `{file, line, severity, category, message, suggestion}`
9. Trajectory recorded: step 2 = LLM analysis
10. **Evaluation** record created: score (via `calculate_score()`), findings, token usage, duration
11. Run status → `"completed"`

Step 4: Synthesis Trigger

Celery Task: `check_and_trigger_synthesis(review_id)`

- Runs after *each* specialist completes — checks if **all 4 specialists are done**
- When all 4 are complete → dispatches `run_synthesis_review`

Step 5: Synthesis

Celery Task: `run_synthesis_review(run_id, review_id)`

1. Collects all findings from the 4 specialist evaluations
2. Calls `AgentReviewer.run_synthesis()` with all specialist results
3. The synthesis LLM merges, deduplicates, and prioritizes findings
4. Creates a unified evaluation with the final score
5. Updates the **Review** record: `overall_score`, `summary`, `findings_count`, `synthesis_findings`
6. Review status → `"completed"`
7. **Notifications dispatched:** Slack and/or Discord webhooks (project-level or global)

Step 6: Baseline (parallel)

Celery Task: `run_baseline_review(run_id, review_id)`

- Runs independently and in parallel with the multi-agent pipeline
- A *single LLM call* with a generic review prompt (no specialization, no static pre-analysis)
- Result is stored for comparison — `baseline_score`, `baseline_findings_count` on the Review record

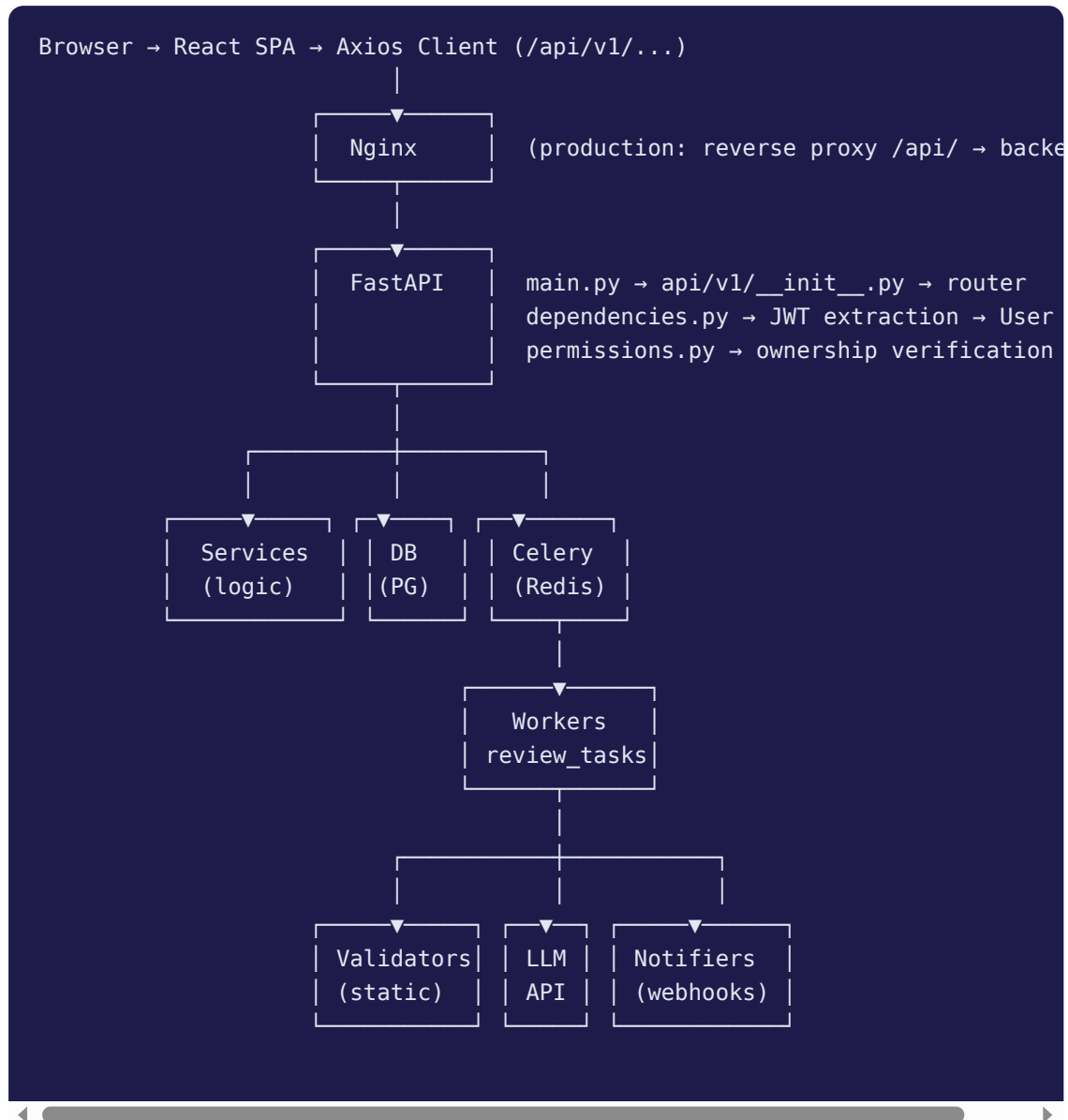
- Enables measuring the improvement of the multi-agent approach vs. a naive single-pass

Step 7: Report Delivery

- Frontend polls via WebSocket or periodic refresh
- `GET /reviews/{id}/report` assembles the full report with findings grouped by agent, scores, and baseline comparison
- User can browse findings by agent tab, view severity badges, and see baseline delta
- Optional: trigger **auto-fix** (sends findings + diff to LLM for code correction suggestions)
- Optional: provide **feedback** (accept/dismiss/false_positive) on each finding

6. Data Flow & Component Interaction

6.1 Request Flow



6.2 Authentication Flow

Email/Password Login:

POST /auth/login → AuthService.login() → verify bcrypt hash → create JWT →

GitHub OAuth Login:

Frontend → github.com/login/oauth/authorize (with client_id, redirect_uri,

GitHub → redirect to /auth/github/callback?code=xxx

Frontend → POST /auth/github/callback {code}

Backend → exchange code for GitHub access_token → get GitHub user → find/c

GitHub Account Connect (existing user):

Frontend → github.com/login/oauth/authorize (with state=connect)

```

GitHub → redirect to /auth/github/callback?code=xxx&state=connect
Frontend → POST /auth/github/connect {code} (with JWT)
Backend → exchange code → link GitHub ID to existing user

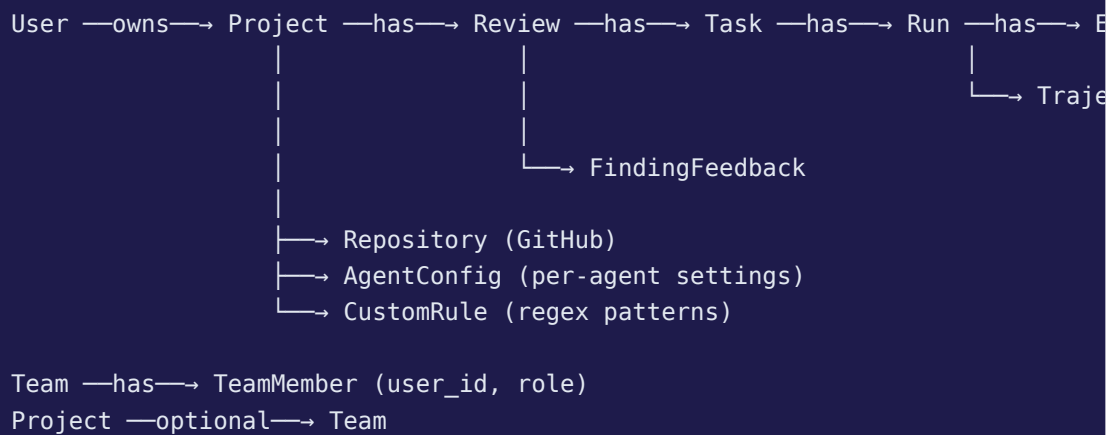
```

6.3 API Key Storage Flow

```
User submits API key → PUT /settings/api-keys
  → encrypt_value(key, SECRET_KEY) → Fernet encryption → store in User.gemini_api_key

Review needs key → ReviewService / Worker
  → decrypt_value(encrypted_key, SECRET_KEY) → plaintext key → pass to LLM prompt
```

6.4 Review Data Relationships



7. Database Schema & Models

Schema Evolution (12 Alembic Migrations)

#	Migration	Changes
1	2ecc38212f50	Initial schema: users, projects, repositories, tasks, runs, evaluations, trajectories, checkpoints
2	fc349d867264	Add GitHub fields to users (github_id, github_username, github_token, avatar_url)
3	1b9eff9b8cf8	Add encrypted Gemini API key to users
4	bda8004e2d38	Add Review model + review_id FK on tasks
5	1fe8f19a7977	Add OpenAI and Anthropic encrypted API key fields to users
6	c02c126a60f1	Add agent_configs table (per-project per-agent configuration)
7	9912e438a340	Add provider and model_name columns to agent_configs
8	b8e6db74d8ee	Add teams, team_members tables + projects.team_id
9	ed30a473ff50	Add Slack/Discord webhook URLs to projects
10	ef774c2041bf	Add custom_rules table
11	88646d2b56c9	Add finding_feedback table

Key Constraints & Indexes

- **users**: Unique on email, username; indexed on github_id
- **teams**: Unique + indexed on slug
- **team_members**: Unique constraint on (team_id, user_id)
- **custom_rules**: Unique constraint on (project_id, name)
- **finding_feedback**: Unique constraint on (review_id, finding_hash)
- **agent_configs**: Unique constraint on (project_id, agent_type)
- All FKs use **CASCADE** delete for child records

8. Authentication & Security

8.1 Authentication Mechanisms

Method	Flow
Email/Password	bcrypt hash stored in DB → JWT issued on login (HS256, 30-min expiry)
GitHub OAuth	OAuth 2.0 authorization code flow → GitHub access token → JWT issued

8.2 Authorization

- **JWT bearer tokens** in `Authorization` header, extracted by `dependencies.py`
- **Ownership checks** via `permissions.py` — every mutating operation verifies the requesting user owns the resource
- **Team roles**: admin, reviewer, viewer — enforced in team endpoints
- **Rate limiting**: Redis sliding window, 10 requests/hour for review creation

8.3 Secrets Management

- **API keys** (Gemini, OpenAI, Anthropic) are encrypted at rest using Fernet symmetric encryption
- Encryption key derived from `SECRET_KEY` via SHA-256 hash
- **GitHub tokens** stored in the database (used for API access on behalf of users)
- **Webhook secrets** validated via HMAC-SHA256 for GitHub webhook payloads
- **.env files** are gitignored; `.env.example` files provide templates

8.4 Security Defenses

- CORS configured for known origins
- Automatic 401 redirect on expired/invalid tokens (frontend interceptor)
- Diff size limit (`MAX_DIFF_SIZE`) prevents abuse
- Rate limiting on review creation to prevent resource exhaustion

9. GitHub Integration

9.1 Features

- **OAuth Login:** Sign in with GitHub (creates/links account)
- **Account Connection:** Link GitHub to an existing email/password account
- **Repository Browsing:** List user's GitHub repos, connect them to projects
- **PR Review:** List open PRs, fetch diffs, trigger reviews directly from PRs
- **Comment Posting:** Post review results as PR comments on GitHub
- **Webhooks:** Auto-trigger reviews on PR open/synchronize events

9.2 Webhook Flow

```
GitHub → POST /api/v1/github/webhooks
  → Verify HMAC-SHA256 signature
  → Parse event type (pull_request)
  → Check action (opened/synchronize)
  → Find user by GitHub sender ID
  → Find connected repository
  → Fetch PR diff
  → Auto-create review (same as manual submission)
```

9.3 Required Configuration

Variable	Where	Purpose
<code>GITHUB_CLIENT_ID</code>	Backend .env + Frontend .env	OAuth App identification
<code>GITHUB_CLIENT_SECRET</code>	Backend .env only	OAuth App secret (server-side token exchange)
<code>GITHUB_REDIRECT_URI</code>	Backend .env	OAuth callback URL (must match GitHub App settings)
<code>GITHUB_WEBHOOK_SECRET</code>	Backend .env	HMAC signature verification for webhooks

10. Advanced Features

10.1 Auto-Fix Suggestions

After a review completes, users can request auto-fix suggestions via `POST /reviews/{id}/auto-fix`. The `AutoFixer` sends the diff + critical/warning findings to the LLM, which generates corrected code snippets. The frontend shows before/after code blocks with explanations in a dedicated tab.

10.2 Multi-Provider LLM Support

Each agent can be configured independently to use a different LLM provider:

Provider	Default Model	Notes
Gemini	gemini-3.6-flash	Default. Uses JSON response mode for structured output.
OpenAI	gpt-4o-mini	Uses <code>response_format: {"type": "json_object"}</code>
Anthropic	claude-sonnet-4-20250514	Uses tool-use pattern for structured JSON extraction

Users store encrypted API keys via the Settings page. Per-project agent configuration (provider, model, custom prompt, enable/disable) is managed via the Agent Config page.

10.3 Teams & Organizations

Users can create teams with a name and slug. Members are added by user ID with one of three roles: **admin** (full control), **reviewer** (can review), **viewer** (read-only). Projects can optionally belong to a team via `team_id`. The team owner is auto-added as admin and cannot be removed.

10.4 Custom Rule Engine

Per-project regex-based rules that run alongside the built-in static validators. Each rule defines:

- **Pattern**: A regex to match against added lines in the diff
- **File pattern** (optional): Only check files matching this glob/regex
- **Severity**: critical, warning, or info
- **Category & message**: What to report when the pattern matches
- **Suggestion**: Recommended fix

- **Enabled/disabled** toggle

10.5 Finding Feedback & Learning

Users can mark each finding as **accept**, **dismiss**, or **false_positive**. Findings are identified by a SHA-256 hash of (file, line, category, message). The system tracks:

- **Per-category dismiss rates**: Which types of findings users most often disagree with
- **Per-agent accuracy**: What percentage of each agent's findings are accepted

These stats are available on the project feedback stats endpoint and can inform future prompt tuning.

10.6 Notifications

When a review completes, the system sends webhook notifications to:

- **Slack**: Block Kit formatted message with score, severity breakdown, and link
- **Discord**: Embed message with color-coded score (green/yellow/red)

Webhook URLs can be configured globally (env vars) or per-project (in project settings).

10.7 Analytics Dashboard

Provides insights into review activity:

- **Overview**: Total reviews, average score, recent activity
- **Score Trends**: Daily average scores over time (SVG line chart)
- **Category Breakdown**: Most common finding categories (bar chart)
- **Agent Performance**: Per-agent average score, duration, and finding counts

11. DevOps & Deployment

11.1 Docker Compose Services

Service	Image	Port	Role
postgres	postgres:16-alpine	5433:5432	Primary database with health check
redis	redis:7-alpine	6379	Celery broker + rate limiter backend
api	./backend (custom)	8000	FastAPI app. Runs alembic upgrade head then uvicorn
celery-worker	./backend (custom)	—	Celery worker (concurrency=2)
frontend	./frontend (custom)	3000:80	Nginx serving React SPA + API proxy

Startup Order

PostgreSQL and Redis start first (with health checks). The API service waits for both to be healthy, then runs Alembic migrations before starting uvicorn. The Celery worker also waits for both. The frontend depends only on the API being up.

11.2 Dockerfiles

- Backend:** Python 3.12-slim → install requirements → copy app → uvicorn on port 8000
- Frontend:** Multi-stage build. Stage 1: Node 22 → npm ci → vite build . Stage 2: nginx:alpine → copy dist + nginx.conf → serve on port 80

11.3 Nginx Configuration

- location /api/ → reverse proxy to api:8000 (Docker service name)
- location / → serve static files with SPA fallback (try_files \$uri /index.html)

11.4 CI/CD Pipeline (GitHub Actions)

Job	Steps
-----	-------

backend-lint	Python 3.12 → pip install ruff → ruff check .
backend-test	Python 3.12 → install deps → pytest app/tests/ -v
frontend-lint	Node 22 → npm ci → oxlint
frontend-test	Node 22 → npm ci → vitest run
frontend-build	Node 22 → npm ci → vite build (depends on lint+test)
docker-build	docker compose build (depends on backend-test + frontend-build)

Runs on push and PR to main / master branches.

11.5 Environment Variables

Variable	Required	Default
DATABASE_URL	Yes	postgresql://codalyra:codalyra@localhost:5432/codalyra
REDIS_URL	Yes	redis://localhost:6379/0
SECRET_KEY	Yes	change-me-in-production (⚠ must change!)
GEMINI_API_KEY	For reviews	—
GITHUB_CLIENT_ID	For GitHub	—
GITHUB_CLIENT_SECRET	For GitHub	—
VITE_API_BASE_URL	No	/api/v1 (relative, production-ready)
VITE_GITHUB_CLIENT_ID	For GitHub	—

12. Testing Strategy

12.1 Backend Tests (126 tests — pytest)

Test Suite	Coverage
<code>unit/test_auth.py</code>	Registration, login, token generation, duplicate handling
<code>unit/test_projects.py</code>	Project CRUD, ownership verification
<code>unit/test_reviews.py</code>	Review creation, report generation, status transitions
<code>unit/test_tasks.py</code>	Task lifecycle management
<code>unit/test_runs.py</code>	Run state management
<code>unit/test_evaluations.py</code>	Evaluation creation and retrieval
<code>unit/test_trajectories.py</code>	Execution trace recording
<code>unit/test_ai.py</code>	Scoring, deduplication, analyzer utilities
<code>unit/test_validators.py</code>	All static validators (Python, JS, Go, security, language detection)
<code>integration/test_api.py</code>	Full API endpoint testing via TestClient
<code>integration/test_database.py</code>	ORM model relationships and constraints
<code>integration/test_github.py</code>	GitHub service mocked integration tests
<code>integration/test_review_pipeline.py</code>	End-to-end review pipeline with mocked LLM

Test database: SQLite in-memory (via conftest.py) for fast execution without PostgreSQL dependency.

12.2 Frontend Tests (47 tests — Vitest + React Testing Library)

Test Suite	Tests	Coverage
<code>Login.test.jsx</code>	7	Form rendering, submission, error handling, GitHub button, loading state
<code>Dashboard.test.jsx</code>	9	Welcome message, loading, empty state, project rendering, health status, modals
<code>Settings.test.jsx</code>	7	Provider sections, save/active states, encryption notice

<code>AuthContext.test.jsx</code>	5	Token loading, failed auth, login/logout flow
<code>ApiClient.test.js</code>	19	All API module methods (auth, projects, reviews, settings, health, agentConfigs)

Test environment: jsdom with `@testing-library/jest-dom` matchers and Vitest globals.

Total Test Coverage

173 tests (126 backend + 47 frontend) — all passing.